

Stopwatch Component

1. Stopwatch.js - Main Component

jsx

```
import React, { useState, useEffect, useRef } from 'react';
import './Stopwatch.css'; // Optional: for external styles

const Stopwatch = () => {
  // State management
  const [time, setTime] = useState(0);
  const [isRunning, setIsRunning] = useState(false);
  const [isPaused, setIsPaused] = useState(false);
  const [laps, setLaps] = useState([]);

  // Refs for interval and start time
  const intervalRef = useRef(null);
  const startTimeRef = useRef(0);
  const pausedTimeRef = useRef(0);

  // useEffect for timer functionality
  useEffect(() => {
    // Cleanup function to clear interval on unmount
    return () => {
      if (intervalRef.current) {
        clearInterval(intervalRef.current);
      }
    };
  }, []);

  // Effect to handle timer updates
  useEffect(() => {
    if (isRunning && !isPaused) {
      startTimeRef.current = Date.now() - (pausedTimeRef.current || time * 1000);

      intervalRef.current = setInterval(() => {
        const elapsedTime = Math.floor((Date.now() - startTimeRef.current) / 1000);
        setTime(elapsedTime);
      }, 100);
    } else if (intervalRef.current) {
      clearInterval(intervalRef.current);
    }

    return () => {
      if (intervalRef.current) {
        clearInterval(intervalRef.current);
      }
    };
  }, [isRunning, isPaused]);

  // Handler functions
  const handleStart = () => {
    setTime(0);
    setIsRunning(true);
    setIsPaused(false);
    pausedTimeRef.current = 0;
    setLaps([]);
  };

  const handleStop = () => {
    setIsRunning(false);
    setIsPaused(false);
    setTime(0);
    pausedTimeRef.current = 0;
    setLaps([]);
    if (intervalRef.current) {
      clearInterval(intervalRef.current);
    }
  };
};
```

```

    }
  };

  const handlePause = () => {
    if (isRunning && !isPaused) {
      setIsPaused(true);
      pausedTimeRef.current = time;
    }
  };

  const handleResume = () => {
    if (isRunning && isPaused) {
      setIsPaused(false);
    }
  };

  const handleLap = () => {
    if (isRunning && !isPaused) {
      setLaps([...laps, time]);
    }
  };

  const handleReset = () => {
    setIsRunning(false);
    setIsPaused(false);
    setTime(0);
    pausedTimeRef.current = 0;
    setLaps([]);
    if (intervalRef.current) {
      clearInterval(intervalRef.current);
    }
  };

  // Format time function
  const formatTime = (totalSeconds) => {
    const hours = Math.floor(totalSeconds / 3600);
    const minutes = Math.floor((totalSeconds % 3600) / 60);
    const seconds = totalSeconds % 60;

    return `${hours.toString().padStart(2, '0')}:${minutes
      .toString()
      .padStart(2, '0')}:${seconds.toString().padStart(2, '0')}`;
  };

  // Styles object
  const styles = {
    container: {
      display: 'flex',
      justifyContent: 'center',
      alignItems: 'center',
      minHeight: '100vh',
      backgroundColor: '#f0f2f5',
      fontFamily: 'Arial, sans-serif',
      padding: '20px'
    },
    stopwatch: {
      backgroundColor: 'white',
      borderRadius: '20px',
      padding: '40px',
      boxShadow: '0 10px 40px rgba(0,0,0,0.1)',
      textAlign: 'center',
      maxWidth: '600px',
      width: '100%'
    },
    title: {
      color: '#333',
      fontSize: '32px',
      marginBottom: '30px',
      fontWeight: 'bold'
    },
    timeDisplay: {

```

```

    fontSize: '72px',
    fontWeight: 'bold',
    color: '#007bff',
    marginBottom: '30px',
    fontFamily: 'monospace',
    textShadow: '2px 2px 4px rgba(0,0,0,0.1)',
    backgroundColor: '#f8f9fa',
    padding: '20px',
    borderRadius: '10px',
    border: '1px solid #e0e0e0'
  },
  buttonContainer: {
    display: 'grid',
    gridTemplateColumns: 'repeat(3, 1fr)',
    gap: '15px',
    marginBottom: '30px'
  },
  button: {
    padding: '15px 20px',
    fontSize: '18px',
    fontWeight: 'bold',
    border: 'none',
    borderRadius: '10px',
    cursor: 'pointer',
    transition: 'all 0.3s ease',
    color: 'white',
    ':hover': {
      transform: 'translateY(-2px)',
      boxShadow: '0 5px 15px rgba(0,0,0,0.2)'
    },
    ':disabled': {
      opacity: 0.5,
      cursor: 'not-allowed',
      transform: 'none'
    }
  },
  startButton: {
    backgroundColor: '#28a745'
  },
  stopButton: {
    backgroundColor: '#dc3545'
  },
  pauseButton: {
    backgroundColor: '#ffc107',
    color: '#333'
  },
  resumeButton: {
    backgroundColor: '#17a2b8'
  },
  lapButton: {
    backgroundColor: '#6c757d'
  },
  resetButton: {
    backgroundColor: '#343a40'
  },
  status: {
    fontSize: '18px',
    marginBottom: '20px',
    padding: '10px',
    borderRadius: '5px',
    backgroundColor: '#e9ecef'
  },
  lapsContainer: {
    marginTop: '30px',
    textAlign: 'left',
    maxHeight: '200px',
    overflowY: 'auto',
    border: '1px solid #dee2e6',
    borderRadius: '10px',
    padding: '15px'
  },

```

```

    lapsTitle: {
      fontSize: '20px',
      fontWeight: 'bold',
      marginBottom: '15px',
      color: '#495057'
    },
    lapItem: {
      padding: '8px 15px',
      margin: '5px 0',
      backgroundColor: '#f8f9fa',
      borderRadius: '5px',
      display: 'flex',
      justifyContent: 'space-between',
      alignItems: 'center'
    },
    lapNumber: {
      fontWeight: 'bold',
      color: '#6c757d'
    },
    lapTime: {
      fontFamily: 'monospace',
      color: '#28a745'
    }
  }
};

// Get status text
const getStatusText = () => {
  if (!isRunning) return '■ Stopped';
  if (isPaused) return '⏸ Paused';
  return '▶ Running';
};

return (
  <div style={styles.container}>
    <div style={styles.stopwatch}>
      <h1 style={styles.title}>🕒 Stopwatch</h1>

      <div style={styles.status}>Status: {getStatusText()}</div>

      <div style={styles.timeDisplay}>
        {formatTime(time)}
      </div>

      <div style={styles.buttonContainer}>
        <button
          onClick={handleStart}
          style={{...styles.button, ...styles.startButton}}
          disabled={isRunning}
        >
          Start
        </button>

        <button
          onClick={handleStop}
          style={{...styles.button, ...styles.stopButton}}
          disabled={!isRunning}
        >
          Stop
        </button>

        <button
          onClick={handlePause}
          style={{...styles.button, ...styles.pauseButton}}
          disabled={!isRunning || isPaused}
        >
          Pause
        </button>

        <button
          onClick={handleResume}

```

```

        style={{...styles.button, ...styles.resumeButton}}
        disabled={!isPaused}
      >
        Resume
      </button>

      <button
        onClick={handleLap}
        style={{...styles.button, ...styles.lapButton}}
        disabled={!isRunning || isPaused}
      >
        Lap
      </button>

      <button
        onClick={handleReset}
        style={{...styles.button, ...styles.resetButton}}
      >
        Reset
      </button>
    </div>

    {laps.length > 0 && (
      <div style={styles.lapsContainer}>
        <h3 style={styles.lapsTitle}>Lap Times</h3>
        {laps.map((lapTime, index) => (
          <div key={index} style={styles.lapItem}>
            <span style={styles.lapNumber}>Lap {index + 1}</span>
            <span style={styles.lapTime}>{formatTime(lapTime)}</span>
          </div>
        ))}
      </div>
    )}
  </div>
</div>
);
};

export default Stopwatch;

```

2. Alternative Simpler Version (StopwatchSimple.js)

```

jsx

import React, { useState, useEffect } from 'react';

const StopwatchSimple = () => {
  const [time, setTime] = useState(0);
  const [isActive, setIsActive] = useState(false);
  const [isPaused, setIsPaused] = useState(false);

  useEffect(() => {
    let interval = null;

    if (isActive && !isPaused) {
      interval = setInterval(() => {
        setTime((time) => time + 1);
      }, 1000);
    } else {
      clearInterval(interval);
    }

    return () => clearInterval(interval);
  }, [isActive, isPaused]);

  const handleStart = () => {
    setIsActive(true);
    setIsPaused(false);
    setTime(0);
  };
};

```

```

const handleStop = () => {
  setIsActive(false);
  setIsPaused(false);
  setTime(0);
};

const handlePause = () => {
  setIsPaused(true);
};

const handleResume = () => {
  setIsPaused(false);
};

const styles = {
  container: {
    display: 'flex',
    justifyContent: 'center',
    alignItems: 'center',
    height: '100vh',
    backgroundColor: '#282c34'
  },
  stopwatch: {
    backgroundColor: 'white',
    padding: '40px',
    borderRadius: '10px',
    textAlign: 'center'
  },
  time: {
    fontSize: '48px',
    marginBottom: '20px'
  },
  button: {
    padding: '10px 20px',
    margin: '5px',
    fontSize: '16px',
    cursor: 'pointer',
    border: 'none',
    borderRadius: '5px'
  }
};

return (
  <div style={styles.container}>
    <div style={styles.stopwatch}>
      <h1>Stopwatch</h1>
      <div style={styles.time}>{time}s</div>
      <div>
        <button style={styles.button} onClick={handleStart}>Start</button>
        <button style={styles.button} onClick={handleStop}>Stop</button>
        <button style={styles.button} onClick={handlePause}>Pause</button>
        <button style={styles.button} onClick={handleResume}>Resume</button>
      </div>
    </div>
  </div>
);
};

export default StopwatchSimple;

```

3. App.js - Main Application Component

```

jsx

import React from 'react';
import Stopwatch from './Stopwatch';
// import StopwatchSimple from './StopwatchSimple';

function App() {

```

```
return (  
  <div className="App">  
    <Stopwatch />  
    { /* <StopwatchSimple /> */ }  
  </div>  
)  
);  
}  
  
export default App;
```

4. index.js - Entry Point

```
jsx  
  
import React from 'react';  
import ReactDOM from 'react-dom/client';  
import './index.css';  
import App from './App';  
  
const root = ReactDOM.createRoot(document.getElementById('root'));  
root.render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```

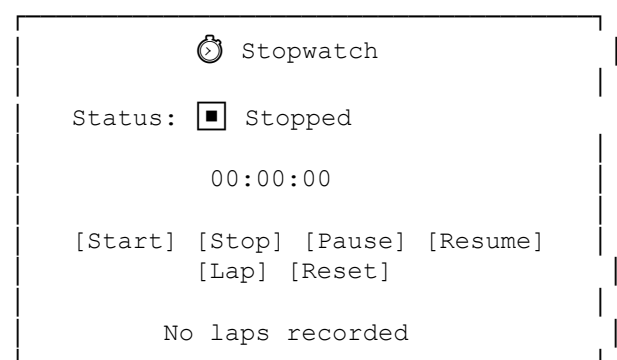
5. index.css - Global Styles

```
css  
  
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}  
  
body {  
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',  
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',  
    sans-serif;  
  -webkit-font-smoothing: antialiased;  
  -moz-osx-font-smoothing: grayscale;  
}  
  
code {  
  font-family: source-code-pro, Menlo, Monaco, Consolas, 'Courier New',  
    monospace;  
}
```

Expected Output

Initial State (Stopped)

text



Running State

text

 Stopwatch

Status:  Running

00:05:23

[Start] [Stop] [Pause] [Resume]

[Lap] [Reset]

Lap Times

Lap 100:02:15

Lap 200:03:45

Lap 300:05:23

Paused State

text

 Stopwatch

Status:  Paused

00:03:45

[Start] [Stop] [Pause] [Resume]

[Lap] [Reset]

Button Functionality Summary

Button	Action	State Change	Time Behavior
Start	Starts from 0	Running = true, Paused = false	Resets to 0, begins counting
Stop	Stops and resets	Running = false, Paused = false	Resets to 0
Pause	Pauses current time	Running = true, Paused = true	Stops incrementing
Resume	Resumes from paused	Running = true, Paused = false	Continues from paused time
Lap	Records current time	No change	Records split time
Reset	Resets everything	Running = false, Paused = false	Resets to 0, clears laps

Key Features Implemented

✔ Time Management:

- Displays time in HH:MM:SS format
- Updates every 100ms for smooth display
- Accurate time tracking using Date object

✔ State Management:

- `isRunning`: Controls if stopwatch is active

- `isPaused`: Controls pause state
- `time`: Current elapsed time
- `laps`: Array of lap times

✓ **Button Controls:**

- **Start**: Begins timing from 0
- **Stop**: Stops and resets to 0
- **Pause**: Temporarily stops counting
- **Resume**: Continues from paused time
- **Lap**: Records current time
- **Reset**: Resets all states

✓ **Cleanup Function:**

- Clears interval on component unmount
- Prevents memory leaks

To Run the Application

1. Create a new React app:

```
bash
```

```
npx create-react-app stopwatch-app  
cd stopwatch-app
```

2. Replace the default files with the code above

3. Start the application:

```
bash
```

```
npm start
```

4. Open <http://localhost:3000> to view it

The stopwatch provides a complete timing solution with all the required functionality plus additional features like lap timing and proper cleanup!